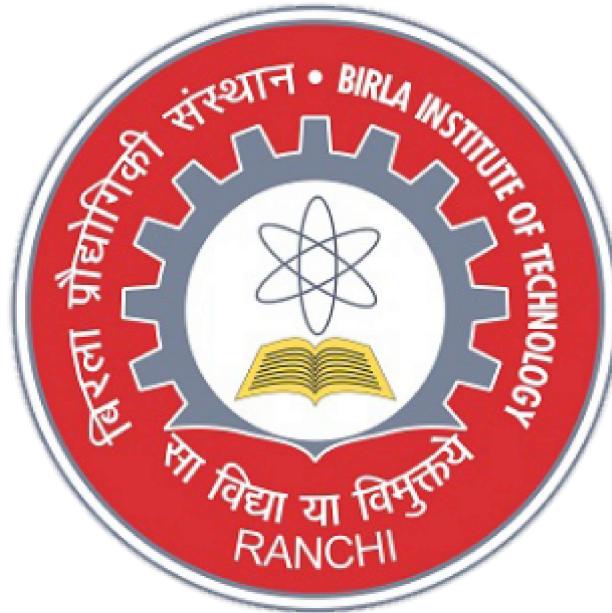


BIRLA INSTITUTE OF TECHNOLOGY

NOIDA OFF CAMPUS, MESRA



PRACTICAL FILE

MACHINE LEARNING

NAME: Pulkit Shandilya

ROLL NO.: BCA/45003/23

Department: Computer Science [CSE]

VISION

The department strives to be recognized for outstanding education and research, leading to excellent professionals and innovators in the field of Computer Science and Engineering, who can positively contribute to the society.

MISSION

The mission of the department is to impart quality education and research, to produce innovative and globally competent technocrats as well as young researchers having sound knowledge in the fields of Computer Science & Engineering.

1. To impart quality education and equip the students with a strong foundation that could make them capable of handling challenges of the new century.
2. To maintain state of the art research facilities and facilitate interaction with world's leading universities, industries and research organisations for constant improvement in the quality of education and research.

1. Display Multiplication Table

Write a program to display the multiplication table of a given number up to 10.

```
In [1]: # Multiplication table function
def multiplication_table(number, limit=10):
    print(f"Multiplication table for {number}:")
    print("-" * 25)
    for i in range(1, limit + 1):
        result = number * i
        print(f"{number} x {i} = {result}")

# Test the function
num = 7
multiplication_table(num)
```

Multiplication table for 7:

```
-----
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

2. Factorial with Recursion

Calculate the factorial of a number using recursive function.

```
In [2]: # Recursive factorial function
def factorial_recursive(n):
    # Base case
    if n == 0 or n == 1:
        return 1
    else:
        # Recursive case
        return n * factorial_recursive(n - 1)

# Test the function
numbers = [5, 6, 8, 10]
for num in numbers:
    result = factorial_recursive(num)
    print(f"Factorial of {num} = {result}")
```

```
Factorial of 5 = 120
Factorial of 6 = 720
Factorial of 8 = 40320
Factorial of 10 = 3628800
```

3. Count Vowels in a String

Write a function to count the number of vowels in a given string.

```
In [3]: # Function to count vowels
def count_vowels(text):
    vowels = "aeiouAEIOU"
    count = 0
    vowel_list = []

    for char in text:
        if char in vowels:
            count += 1
            vowel_list.append(char)

    return count, vowel_list

# Test the function
test_strings = [
    "Hello World",
    "Python Programming",
    "Data Science and Machine Learning",
    "Artificial Intelligence"
]

for text in test_strings:
    count, vowels_found = count_vowels(text)
    print(f"Text: '{text}'")
    print(f"Vowel count: {count}")
    print(f"Vowels found: {vowels_found}")
    print("-" * 40)
```

```
Text: 'Hello World'
Vowel count: 3
Vowels found: ['e', 'o', 'o']
-----
Text: 'Python Programming'
Vowel count: 4
Vowels found: ['o', 'o', 'a', 'i']
-----
Text: 'Data Science and Machine Learning'
Vowel count: 12
Vowels found: ['a', 'a', 'i', 'e', 'e', 'a', 'a', 'i', 'e', 'e', 'a', 'i']
-----
Text: 'Artificial Intelligence'
Vowel count: 10
Vowels found: ['A', 'i', 'i', 'i', 'a', 'I', 'e', 'i', 'e', 'e']
-----
```

4. Perform Operations on a List

Demonstrate various list operations including adding, removing, sorting, and filtering elements.

```
In [4]: # Create a sample List
numbers = [12, 45, 7, 23, 56, 89, 34, 67, 8, 90, 15]
print(f"Original list: {numbers}")
print()

# 1. Add elements
numbers.append(100)
numbers.extend([105, 110])
print(f"After adding elements: {numbers}")

# 2. Remove elements
numbers.remove(7) # Remove specific value
removed_item = numbers.pop() # Remove last element
print(f"After removing 7 and last element ({removed_item}): {numbers}")

# 3. Sort the List
sorted_asc = sorted(numbers)
sorted_desc = sorted(numbers, reverse=True)
print(f"Sorted ascending: {sorted_asc}")
print(f"Sorted descending: {sorted_desc}")

# 4. Filter operations
even_numbers = [num for num in numbers if num % 2 == 0]
odd_numbers = [num for num in numbers if num % 2 != 0]
numbers_gt_50 = [num for num in numbers if num > 50]

print(f"Even numbers: {even_numbers}")
print(f"Odd numbers: {odd_numbers}")
print(f"Numbers greater than 50: {numbers_gt_50}")

# 5. Statistical operations
print(f"\nList Statistics:")
print(f"Length: {len(numbers)}")
print(f"Sum: {sum(numbers)}")
print(f"Average: {sum(numbers)/len(numbers):.2f}")
print(f"Maximum: {max(numbers)}")
print(f"Minimum: {min(numbers)}")
```

Original list: [12, 45, 7, 23, 56, 89, 34, 67, 8, 90, 15]

After adding elements: [12, 45, 7, 23, 56, 89, 34, 67, 8, 90, 15, 100, 105, 110]

After removing 7 and last element (110): [12, 45, 23, 56, 89, 34, 67, 8, 90, 15, 100, 105]

Sorted ascending: [8, 12, 15, 23, 34, 45, 56, 67, 89, 90, 100, 105]

Sorted descending: [105, 100, 90, 89, 67, 56, 45, 34, 23, 15, 12, 8]

Even numbers: [12, 56, 34, 8, 90, 100]

Odd numbers: [45, 23, 89, 67, 15, 105]

Numbers greater than 50: [56, 89, 67, 90, 100, 105]

List Statistics:

Length: 12

Sum: 644

Average: 53.67

Maximum: 105

Minimum: 8

5. Check Prime Numbers

Write a function to check if a number is prime and find all prime numbers within a given range.

```
In [5]: # Function to check if a number is prime
def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    # Check odd divisors up to sqrt(n)
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True

# Function to find all prime numbers in a range
def find_primes_in_range(start, end):
    primes = []
    for num in range(start, end + 1):
        if is_prime(num):
            primes.append(num)
    return primes

# Test individual numbers
test_numbers = [2, 17, 25, 29, 31, 49, 53, 97, 100]
print("Prime number checks:")
for num in test_numbers:
    result = is_prime(num)
    print(f"{num} is {'prime' if result else 'not prime'}")

print("\n" + "="*40)
```

```
# Find primes in ranges
ranges = [(1, 20), (50, 70), (90, 110)]
for start, end in ranges:
    primes = find_primes_in_range(start, end)
    print(f"Prime numbers between {start} and {end}: {primes}")
    print(f"Count: {len(primes)}")
    print("-" * 30)
```

Prime number checks:

2 is prime
 17 is prime
 25 is not prime
 29 is prime
 31 is prime
 49 is not prime
 53 is prime
 97 is prime
 100 is not prime

```
=====
Prime numbers between 1 and 20: [2, 3, 5, 7, 11, 13, 17, 19]
Count: 8
-----
Prime numbers between 50 and 70: [53, 59, 61, 67]
Count: 4
-----
Prime numbers between 90 and 110: [97, 101, 103, 107, 109]
Count: 5
-----
```

6. Dictionary Operations and Student Grade Management

Create a student grade management system using dictionaries to demonstrate various dictionary operations.

```
In [6]: # Student Grade Management System using dictionaries
students = {
    "Alice": {"Math": 85, "Science": 92, "English": 78},
    "Bob": {"Math": 92, "Science": 87, "English": 84},
    "Charlie": {"Math": 78, "Science": 90, "English": 88},
    "Diana": {"Math": 95, "Science": 89, "English": 91}
}

print("Original Student Data:")
for student, grades in students.items():
    print(f"{student}: {grades}")
print()

# 1. Add a new student
students["Eve"] = {"Math": 88, "Science": 85, "English": 90}
print("After adding Eve:")
print(f"Eve: {students['Eve']}")
```

```

print()

# 2. Update grades for an existing student
students["Alice"]["History"] = 82 # Add new subject
students["Bob"]["Math"] = 95      # Update existing grade
print("After updates:")
print(f"Alice: {students['Alice']}")
print(f"Bob: {students['Bob']}")
print()

# 3. Calculate average grade for each student
print("Student Averages:")
student_averages = {}
for student, grades in students.items():
    average = sum(grades.values()) / len(grades.values())
    student_averages[student] = round(average, 2)
    print(f"{student}: {average:.2f}")
print()

# 4. Find subject averages
subjects = set()
for grades in students.values():
    subjects.update(grades.keys())

print("Subject Averages:")
for subject in subjects:
    total = 0
    count = 0
    for student, grades in students.items():
        if subject in grades:
            total += grades[subject]
            count += 1
    if count > 0:
        average = total / count
        print(f"{subject}: {average:.2f}")
print()

# 5. Find top student and top performer in each subject
top_student = max(student_averages, key=student_averages.get)
print(f"Top Overall Student: {top_student} (Average: {student_averages[top_student]})")
print()

print("Top Performers by Subject:")
for subject in sorted(subjects):
    best_grade = 0
    best_student = ""
    for student, grades in students.items():
        if subject in grades and grades[subject] > best_grade:
            best_grade = grades[subject]
            best_student = student
    if best_student:
        print(f"{subject}: {best_student} ({best_grade})")

# 6. Dictionary comprehension examples
print("\nDictionary Comprehensions:")
# Students with average above 85

```

```

high_performers = {student: avg for student, avg in student_averages.items() if avg
print(f"High performers (>85 average): {high_performers}")

# Math grades only
math_grades = {student: grades["Math"] for student, grades in students.items() if "
print(f"Math grades: {math_grades}")

```

Original Student Data:

Alice: {'Math': 85, 'Science': 92, 'English': 78}
 Bob: {'Math': 92, 'Science': 87, 'English': 84}
 Charlie: {'Math': 78, 'Science': 90, 'English': 88}
 Diana: {'Math': 95, 'Science': 89, 'English': 91}

After adding Eve:

Eve: {'Math': 88, 'Science': 85, 'English': 90}

After updates:

Alice: {'Math': 85, 'Science': 92, 'English': 78, 'History': 82}
 Bob: {'Math': 95, 'Science': 87, 'English': 84}

Student Averages:

Alice: 84.25
 Bob: 88.67
 Charlie: 85.33
 Diana: 91.67
 Eve: 87.67

Subject Averages:

Science: 88.60
 History: 82.00
 Math: 88.20
 English: 86.20

Top Overall Student: Diana (Average: 91.67)

Top Performers by Subject:

English: Diana (91)
 History: Alice (82)
 Math: Bob (95)
 Science: Alice (92)

Dictionary Comprehensions:

High performers (>85 average): {'Bob': 88.67, 'Charlie': 85.33, 'Diana': 91.67, 'Eve': 87.67}
 Math grades: {'Alice': 85, 'Bob': 95, 'Charlie': 78, 'Diana': 95, 'Eve': 88}